

Understanding Public and Private Keys

Public key : shared with anyone

Private key : kept by owner

What is a public key

- It's a mathematical value that is one half of a **public-private key pair** used in asymmetric cryptography
- It's a cryptographic value generated using algorithms like RSA, Elliptic Curve Cryptography
- The private key can not be derived from the public key because it is computationally really expensive.

How we use it :

1. For Secure communication Use cases:
 - a. Sender encrypts data using the recipient's **public key**
 - b. Only the recipient can decrypt it using their **private key**
2. For Authentication & Identity use case
 - a. Sender signs data with their **private key**
 - b. Anyone verifies using the sender's **public key**

Digital Signature

- Sign → private key
- Verify → public key

Encryption

- Encrypt → recipient's public key
- Decrypt → recipient's private key

<u>Goal</u>	<u>Operation</u>	<u>Sender Uses</u>	<u>Receiver Uses</u>
Keep data secret	Encryption	Receiver public key	Receiver's private key
Prove identity	Signature	Sender's private key	Sender's public key

Where Public Keys Are Used

1. HTTPS / Web Security
 - a. Server sends public key (certificate)
 - b. Client verifies it (CA trust)
 - c. Secure session established
 - d. Now Browsers can use public keys in TLS certificates
2. SSH (Remote Access)
 - a. You store your **public key on a server**

- b. Authenticate using your private key
 - c. Passwordless login
- 3. Code Signing & Software Updates
 - a. OS verifies updates using developer's public key
 - b. Microsoft Windows updates, Apple iOS/macOS apps

How authentication works

1. Signing (done by sender)
 - The sender uses their private key
 - This creates a digital signature
2. Verification (done by receiver)
 - The receiver uses the sender's public key
 - This checks whether the signature is valid

Signature and validation during authentication

The signature

1. Take message M
2. Compute hash: $H = \text{hash}(M)$
3. Sign hash using private key $\rightarrow S = \text{Sign}(H, \text{private_key})$
4. Output = (M, S)

The validation

1. Receiver gets (M, S)
2. Compute hash again: $H1 = \text{hash}(M)$
3. Verify signature using public key $\rightarrow H2 = \text{Verify}(S, \text{public_key})$
4. [Inference]
 - a. If $H1 == H2 \rightarrow$ valid (authentic + unmodified)
 - b. Else $\rightarrow$ invalid (tampered or fake)

JWT Signing

A JSON Web Token (JWT) is: `header.payload.signature`

- **header** $\rightarrow$ algorithm (e.g., HS256, RS256)
- **payload** $\rightarrow$ claims (user, role, etc.)
- **signature** $\rightarrow$ proves integrity & authenticity

Case 1: Symmetric (HMAC)

- Algorithm: HS256
- Signing key = same secret for both sides
- signature = $\text{HMAC}(\text{secret}, \text{header} + \text{payload})$

Case 2: Asymmetric

- Algorithm: RS256 (uses RSA)
- Server signs with private key
- Server verifies with public key

Private key → signing
Public key → verification

JWT Attacks

1. "alg=none" Attack with signature stripped

- Modify header: {
 "alg": "none"
}
- If server:
 - Trusts header blindly
 - Skips signature verification

You can forge arbitrary tokens: {"user": "admin"}

2. HMAC ↔ RSA Confusion

- Issue : Server expects RS256 but **accepts HS256**
 - I. Obtain server's **public key** (often exposed)
 - II. Change JWT header: {"alg": "HS256"}
 - III. Use **public key as HMAC secret**:
 - A. signature = HMAC(public_key, header + payload)

3. Signature Not Verified Properly

- issues : Ignoring signature result
 - Using wrong key
 - Accepting expired tokens
 - I. {"role": "admin"}
 - II. Keep old signature → still accepted

4. Weak or hard coded secretSecret (HS256)

- Common culprits found in real apps:
 - Secret
 - Password
 - 123456
 - your-256-bit-secret ← literal from jwt.io docs
 - change_this_in_production ← never changed
 - "" ← empty string accepted
 - Brute-force secret
 - Using tools like hashcat , jwt_tool
 - hashcat -a 0 -m 16500 token.jwt wordlist.txt
 - Recover secret → sign arbitrary tokens

5. Key Injection via kid Header

- SQL Injection via kid:
 - { "alg": "HS256", "kid": "' UNION SELECT 'attacker_secret' -- "
- Path Traversal via kid: Server loads key from filesystem
 - { "kid": "../../../dev/null" }
 - { "alg": "RS256", "kid": "../../../etc/passwd" }
- We Fetch key from attacker-controlled URL

6. No Token Revocation

- Since JWTs are **stateless**, servers that don't maintain a denylist have **no real logout**.
- Attacker with a stolen token can use it after the victim "logs out", after a password reset, after account deactivation. Very common in SPAs and mobile backends.

7. jku / x5u Header Injection

- The jku (JWK Set URL) header points to where the public key lives. Developers who trust this header are vulnerable.
- { "alg": "RS256", "jku": "https://attacker.com/evil-jwks.json" }
- Attacker hosts their own JWKS, signs a token with their private key, server fetches attacker's URL and uses attacker's public key to verify → **attacker-signed tokens are trusted**.

8. jwk Header Embedding

- the attacker **embeds their own public key directly in the token header**:

```
{  "alg": "RS256",  "jwk": {    "kty": "RSA",    "n": "attacker_public_key_modulus...",    "e": "AQAB"  }}
```
- Server uses the embedded key to verify – attacker forges any claims they want. This is a **CVE-class issue** that surfaced repeatedly in 2023-2024 across enterprise SSO implementations.

9. JWT Confusion with Session Cookies

Modern developer misunderstanding and their consequences

1. JWT payload is secure

- a. Misunderstanding : "JWT is encrypted, so data inside is safe"
- b. Reality : JWT (JWS) is only base64 encoded, not encrypted , Anyone can decode it instantly
- c. If Sensitive data visible (emails, roles, API keys) information leak

2. If it's signed, it's safe to trust

- a. Misunderstanding : "Valid signature = safe to use all claims"
- b. Reality : Signature = **integrity**, NOT correctness
 - i. The server must still validate claims (**role**, **scope**, etc.)

Example 1 : You sign up for a new account. The app sends a request to the server:

- POST /register { "username": "John", "role": "user" }
- An attacker intercepts the request and changes it to: { "username": "John", "role": "admin" }
- The server takes that "admin" string, sticks it into a token, and signs it
- The attacker now has a **perfectly valid signature** from the server that says they are an admin. When they show this token to other parts of the app, those parts see the valid signature

Example 2 : You go to your settings to change your display name. The app sends:

- PATCH /update-profile { "display_name": "Johnny" }
- The attacker adds a hidden field: { "display_name": "Johnny", "is_premium_subscriber": true }
- If the server-side code is written lazily (e.g., `user.update(request.body)`), it might save that "premium" status to the database. Then, it generates a new signed token to "refresh" your session.

Example 3 : You log into an App. The server verifies your password and gives you a signed token. This token is meant only to let you download your books:

- You don't have an admin password, but you decide to send your App's token to the Admin Console's login gate.

- The Admin Console is programmed poorly. It only checks the signature: if `(isValidSignature(token)) { grantAccess(); }`. It sees that the token was signed by the official Server, so it assumes the token is "good" for anything.

3. Algorithm in header is trustworthy

- a. Misunderstanding : JWT header tells us how to verify it
- b. Reality : Header is attacker-controlled input
Accepting arbitrary algorithms = critical vuln
- c. Switch RS256 → HS256 and Use public key as HMAC secret.

4. Signature verification is always done correctly

- a. Misunderstanding : Libraries handle everything safely.
- b. Reality : Many systems:
 - a. Skip verification
 - b. Use weak checks
 - c. Accept invalid signatures
- c. Modify payload (role=admin) → Remove or tamper signature

5. Public key crypto = authorization

- a. Misunderstanding : If token is signed, user is allowed
- b. Reality : signed token only proves Who issued token not whether user should have access.

- i. Check if user has the right to see the resource

Example 1 : You are logged in as "User A."

You have a perfectly valid signed token

- You send a request to GET `/user-messages?id=UserB`. You include your valid "User A" token
- The server checks the signature. It then fetches the messages for **User B** because the code didn't check if User A had the right to see User B's data.
- You just read someone else's private messages

6. Secrets don't need to be strong

- a.